

MANUAL COMPLETO DE PROGRAMACIÓN I

Grado en Ingeniería Informática - 1º Curso

Este documento unifica todos los temas del plan de estudio de Fundamentos y Programación Estructurada en C++ en un único manual para facilitar su lectura, impresión o conversión a formatos como PDF.

Índice General

- **Bloque I: Fundamentos y Control de Flujo**
 - [Tema 1: Algoritmos y Programas](#)
 - [Tema 2: Tipos de Datos y Expresiones](#)
 - [Tema 3: Instrucciones de Control de Flujo](#)
- **Bloque II: Diseño Modular y Estructuras de Datos**
 - [Tema 4: Desarrollo Modular](#)
 - [Tema 5: Vectores y Matrices \(Arrays\)](#)
 - [Tema 6: Cadenas de Caracteres](#)
 - [Tema 7: Registros \(Structs\)](#)
- **Bloque III: Entrada/Salida y Persistencia**
 - [Tema 8: Flujos y Ficheros](#)
- **Secciones Finales**
 - [Glosario de Términos](#)
 - [Bibliografía Recomendada](#)

Tema 1: Algoritmos y Programas

Un programa de ordenador es la plasmación física de un algoritmo mediante un lenguaje estructurado. Este tema introduce los conceptos fundamentales de la arquitectura del ordenador, el proceso de traducción de código fuente a binario ejecutable y el ciclo de vida del software para la construcción de sistemas estables y mantenibles.

1. Introducción a la Informática y Arquitectura de Computadores

El funcionamiento interno de cualquier ordenador moderno sigue el modelo propuesto en 1945 por John von Neumann.

Arquitectura de Von Neumann

La **Arquitectura de Von Neumann** se divide en cuatro bloques principales interconectados por canales de comunicación denominados **buses**:

- 1. Unidad Central de Procesamiento (CPU):** Es el "cerebro" y contiene:
 - **Unidad de Control (UC):** Decodifica y coordina la ejecución de las instrucciones de programa.
 - **Unidad Aritmético-Lógica (ALU):** Realiza operaciones aritméticas (+, -, ·) y lógicas (AND, OR, comparaciones).
 - **Registros:** Memoria interna ultrarrápida de almacenamiento temporal de datos e instrucciones en ejecución.
- 2. Memoria Principal (RAM):** Almacena de forma temporal tanto los datos de entrada/salida como las instrucciones del programa en ejecución. Es volátil (se borra al apagar el equipo).
- 3. Dispositivos de Entrada/Salida (E/S):** Permiten la comunicación con el exterior (teclado, ratón, pantalla, discos duros).
- 4. Buses:** Vías de comunicación físicas (cables o pistas de circuito integrado) que transfieren datos, direcciones de memoria e instrucciones de control entre los componentes.

2. Traductores de Lenguaje: Compiladores vs Intérpretes

Los ordenadores solo comprenden el **lenguaje máquina** (código binario formado por secuencias de 0s y 1s que corresponden a voltajes eléctricos). Para programar a alto nivel (cerca del lenguaje humano), requerimos herramientas que traduzcan el código a binario.

2.1 Compiladores (Ejemplo: C++, Rust, Go)

Traducen todo el código fuente del programa de una sola vez y generan un archivo binario independiente y ejecutable (**.exe** o sin extensión en UNIX).

- **Ventajas:** Ejecución extremadamente rápida (ya traducido al procesador) y protección del código fuente original.
- **Desventajas:** El proceso de compilación toma tiempo y el binario generado no es portátil (un binario compilado en Windows no funciona en Linux o macOS).

2.2 Intérpretes (Ejemplo: Python, Ruby, PHP)

Traducen y ejecutan el código fuente línea a línea en tiempo de ejecución mediante un programa especial denominado intérprete.

- **Ventajas:** Desarrollo rápido y código portátil (el mismo archivo **.py** corre en cualquier SO con el intérprete instalado).
- **Desventajas:** Ejecución mucho más lenta y necesidad de distribuir el código fuente.

3. Herramientas de Desarrollo y Proceso de Compilación en C++

Para construir programas ejecutables en C++, el código pasa por cuatro etapas automáticas:

```
Código Fuente (.cpp)  ==> [ Preprocesador ] ==> Código Expandido
                        ||
Código Binario Objeto (.o) <== [ Compilador ] <====++
                        ||
                        vv
                        [ Enlazador / Linker ] ==> Archivo Ejecutable (.exe o binario)
```

1. **Preprocesador:** Procesa las directivas que inician con **#** (como **#include**), copiando los archivos de cabecera al código fuente.
2. **Compilador:** Convierte el código expandido en lenguaje de bajo nivel (ensamblador) y luego a código binario objeto (archivos **.o** o **.obj**).
3. **Enlazador (Linker):** Junta todos los archivos objeto generados por el compilador junto con las librerías precompiladas del sistema para crear el ejecutable definitivo.
4. **Depurador (Debugger):** Herramienta que permite ejecutar el programa línea a línea para inspeccionar el estado de las variables y detectar errores lógicos de ejecución.

4. Ciclo de Vida del Software

La ingeniería del software define el **Ciclo de Vida del Software** como el conjunto de fases por las que pasa un programa desde su concepción hasta su retirada:

1. **Análisis de Requisitos:** Fase de recolección de las necesidades del cliente (qué debe hacer el programa).
2. **Diseño:** Planificación técnica de la arquitectura del programa (diagramas, bases de datos y algoritmos).
3. **Codificación (Implementación):** Escritura del código fuente en el lenguaje de programación elegido.
4. **Pruebas (Testing):** Validación del código:
 - *Pruebas Unitarias:* Verifican que funciones o módulos específicos actúan correctamente por separado.
 - *Pruebas de Integración:* Verifican que los módulos funcionan correctamente al unirse.

5. **Mantenimiento:** Corrección de errores tras el despliegue (bugs) y adición de nuevas funcionalidades.
-

5. El Toque Informático

Algoritmos en Pseudocódigo

Un **algoritmo** es una secuencia finita, ordenada e inequívoca de pasos para resolver un problema. El **pseudocódigo** es un lenguaje de diseño algorítmico que imita las estructuras lógicas de programación sin importar la sintaxis rígida de un compilador.

A continuación, planteamos un algoritmo básico para calcular el **factorial** de un número entero no negativo n ($n! = n \cdot (n - 1) \dots 1$) en pseudocódigo y mostramos su transcripción directa a código funcional C++.

Algoritmo en Pseudocódigo:

```
Algoritmo Factorial
  Leer n
  Si n < 0 Entonces
    Escribir "El factorial no está definido para negativos"
  Sino
    factorial <- 1
    Para i <- 1 Hasta n Con Paso 1 Hacer
      factorial <- factorial * i
    FinPara
    Escribir "El factorial es: ", factorial
  FinSi
FinAlgoritmo
```

Código Transcrito en C++:

```
#include <iostream>

int main() {
  int n;
  std::cout << "Introduce un entero no negativo: ";
  std::cin >> n;

  if (n < 0) {
    std::cout << "El factorial no esta definido para negativos." << std::endl;
```

```
    } else {  
        long long factorial = 1; // Usamos long long para evitar desbordamiento  
rápido  
        for (int i = 1; i <= n; ++i) {  
            factorial = factorial * i;  
        }  
        std::cout << "El factorial de " << n << " es: " << factorial << std::endl;  
    }  
    return 0;  
}
```

6. Ejercicios Resueltos

Ejercicio 1

Describir brevemente la diferencia entre un error de compilación (sintáctico), un error de ejecución (runtime error) y un error lógico en el desarrollo de software.

Solución:

1. **Error de Compilación:** Ocurre cuando el código viola las reglas gramaticales del lenguaje (por ejemplo, omitir un punto y coma ; o declarar mal una variable). El compilador lo detecta y detiene el proceso de traducción, impidiendo la generación del ejecutable.
2. **Error de Ejecución (Runtime Error):** El programa compila correctamente y genera el binario, pero durante su ejecución realiza una operación prohibida por el sistema operativo (por ejemplo, intentar dividir un número por cero o acceder a una posición de memoria no asignada/puntero nulo). El programa se aborta abruptamente (crash).
3. **Error Lógico:** El programa compila y corre hasta el final sin colapsar, pero produce resultados incorrectos (por ejemplo, calcular el factorial usando suma en lugar de multiplicación). Son los errores más difíciles de detectar y requieren de depuración (debugging) o suite de pruebas unitarias.

7. Ejercicios Propuestos

1. Escribir el pseudocódigo de un algoritmo que lea tres números reales del usuario y calcule y muestre el promedio de los mismos.
2. ¿Qué componente de la arquitectura de Von Neumann se encarga de realizar la suma de dos números enteros a nivel de hardware?
3. Explicar detalladamente por qué las directivas que inician con `#include` en C++ no son procesadas por el compilador propiamente dicho, sino por el preprocesador.

Tema 2: Tipos de Datos y Expresiones

Los ordenadores representan la información del mundo real en forma de bits estructurados. Para operar con ellos de forma segura y eficiente, los lenguajes fuertemente tipados como C++ asocian cada variable a un tipo de dato específico, lo que define cuánta memoria ocupa y qué operaciones son válidas.

1. Tipos de Datos Primitivos y Almacenamiento en Memoria

C++ clasifica sus tipos de datos básicos según el tipo de información y los bytes que ocupan en la memoria RAM:

Tipo	Descripción	Tamaño típico (bytes)	Rango de valores
<code>bool</code>	Valor lógico booleano	1	<code>true</code> (1) o <code>false</code> (0)
<code>char</code>	Carácter individual (ASCII)	1	-128 a 127 o 0 a 255
<code>int</code>	Número entero con signo	4	$-2.14 \cdot 10^9$ a $2.14 \cdot 10^9$
<code>float</code>	Real en coma flotante (precisión simple)	4	$\pm 3.4 \cdot 10^{38}$ (6 decimales)
<code>double</code>	Real en coma flotante (precisión doble)	8	$\pm 1.7 \cdot 10^{308}$ (15 decimales)

El operador `sizeof`: Permite consultar de forma dinámica cuántos bytes asigna el compilador de tu arquitectura a un determinado tipo de dato (por ejemplo, `sizeof(int)`).

2. Variables, Constantes y Ámbito (Alcance)

- **Variables:** Espacios con nombre en la memoria RAM cuyo valor puede alterarse durante la ejecución. Deben declararse indicando su tipo y nombre:

```
int edad = 18;
```

- **Constantes (`const`):** Variables cuyo valor es inalterable tras su inicialización. Intentar modificar una constante produce un error de compilación:

```
const double PI = 3.14159265;
```

- **Ámbito (Scope):** Región del programa donde una variable es visible y accesible.
 - *Locales:* Declaradas dentro de una función o bloque `{ }`. Solo existen durante la ejecución de ese bloque.
 - *Globales:* Declaradas fuera de cualquier función. Son accesibles por todo el código, lo cual se considera una mala práctica porque dificulta la depuración.

3. Operadores y Precedencia

Los operadores combinan variables y constantes para evaluar expresiones:

1. **Aritméticos:** Suma (+), resta (-), multiplicación (*), división (/), resto de división o módulo (%) - solo para enteros).
2. **Relacionales (Comparación):** Igual a (==), diferente de (!=), mayor que (>), menor que (<), mayor o igual (>=), menor o igual (<=).
3. **Lógicos:** Y lógico (&&), O lógico (||), negación lógica (!).
4. **Asignación y Atajos:** Asignación ordinaria (=), y acumuladores combinados (+=, -=, *=, /=).
5. **Incremento / Decremento:** Pre-incremento (++x), post-incremento (x++).

Precedencia de Operadores

Determina el orden en que se evalúan las operaciones matemáticas cuando hay múltiples operadores en una misma línea:

1. Paréntesis () (máxima prioridad)

2. Multiplicación, división y módulo `*`, `/`, `%`
 3. Suma y resta `+`, `-`
 4. Operadores relacionales
 5. Operadores lógicos (`&&` antes que `||`)
 6. Asignación `=` (mínima prioridad)
-

4. Conversión de Tipos (Casting)

- **Conversión Implícita (Promoción):** El compilador promociona de forma automática tipos más pequeños a más grandes para evitar pérdidas de información (por ejemplo, sumar un `int` y un `double` resulta en un `double`).
- **Conversión Explícita (Casting):** Conversión forzada por el programador. En C++ moderno se realiza mediante `static_cast`:

```
double resultado = static_cast<double>(suma) / cantidad;
```

Atención: Si dividimos dos enteros en C++ (por ejemplo, `5 / 2`), el compilador realiza división entera devolviendo `2`. Si queremos obtener la parte decimal (`2.5`), debemos forzar el casting de al menos uno de los operandos.

5. El Toque Informático

5.1 Desbordamiento Aritmético (Overflow)

Dado que las variables se almacenan en un número fijo de bits, existe un valor máximo representable. Si incrementamos una variable que ya se encuentra en su límite superior de rango, esta sufre un **desbordamiento** (overflow) y su valor se "da la vuelta" hacia el valor mínimo representable, lo que puede provocar graves fallos en programas bancarios o de control aeronáutico.

A continuación, implementamos un programa en C++ que ilustra la división entera (y cómo evitarla mediante casting explícito) junto con un desbordamiento de tipo entero de rango corto (`short`).

```
#include <iostream>

int main() {
    // 1. División entera vs Casting explícito
    int a = 5;
    int b = 2;

    double division_entera = a / b;
    double division_correcta = static_cast<double>(a) / b;

    std::cout << "--- División de enteros ---" << std::endl;
    std::cout << "Resultado de " << a << " / " << b << " (sin cast): " <<
division_entera << std::endl;
    std::cout << "Resultado de " << a << " / " << b << " (con cast): " <<
division_correcta << std::endl;

    // 2. Simulación de Desbordamiento Aritmético (Overflow)
    // El tipo 'short' tiene típicamente 16 bits (rango: -32768 a 32767)
    short max_valor = 32767;

    std::cout << "\n--- Simulacion de Overflow ---" << std::endl;
    std::cout << "Valor maximo de short: " << max_valor << std::endl;

    // Sumamos 1 al valor máximo representable
    max_valor = max_valor + 1;

    std::cout << "Valor de short tras sumarle 1 (overflow): " << max_valor <<
std::endl;

    return 0;
}
```

6. Ejercicios Resueltos

Ejercicio 1

Dadas las declaraciones:

```
int x = 10;
int y = 3;
double z = 2.5;
```

Evaluar paso a paso la expresión: $(x / y) + z * 2$

Solución:

1. Analizar la precedencia de operaciones:

- La expresión tiene paréntesis, sumas y productos: $(x / y) + z * 2$.

2. Paso 1: Evaluar los paréntesis (x / y) :

- x e y son de tipo `int`. Por lo tanto, $10 / 3$ realiza una **división entera**, cuyo resultado es 3 (se descarta la parte decimal).

3. Paso 2: Evaluar la multiplicación $z * 2$:

- z es un `double` (2.5). Al multiplicarse por 2 (entero), el 2 se promociona de forma implícita a `double` (2.0).
- $2.5 * 2.0 = 5.0$ (resultado tipo `double`).

4. Paso 3: Evaluar la suma final $3 + 5.0$:

- Sumamos el entero 3 (de la primera evaluación) con el `double` 5.0 .
- El entero 3 se promociona a 3.0 .
- $3.0 + 5.0 = 8.0$.

5. Resultado: El valor final evaluado de la expresión es 8.0 (tipo `double`).

7. Ejercicios Propuestos

1. Escribir un programa en C++ que lea el radio de un círculo del teclado y calcule y muestre su área. (Usa constantes para definir el valor de $\pi = 3.14159$).
2. ¿Qué diferencia hay en el comportamiento del operador módulo (%) frente a la división tradicional (/) en C++?
3. Si declaramos una variable de tipo `char letra = 'A'`; y luego le sumamos 1 (`letra = letra + 1;`), ¿qué valor contendrá la variable al imprimirla en pantalla como carácter? Explica la relación con la tabla ASCII.

Tema 3: Instrucciones de Control de Flujo

Por defecto, los programas ejecutan sus instrucciones en un flujo secuencial estricto (una tras otra en el orden en que están escritas). Las instrucciones de control de flujo permiten romper esta secuencialidad para tomar decisiones en base a condiciones variables y repetir bloques de instrucciones sin duplicar código.

1. Estructuras Condicionales (Selección)

Las condicionales desvían el flujo del programa por diferentes ramificaciones de ejecución basándose en la evaluación de expresiones booleanas (**true** o **false**).

1.1 Estructura **if / if-else**

Permite tomar una bifurcación binaria.

```
if (condicion) {  
    // Código ejecutado si la condición es verdadera (true)  
} else {  
    // Código ejecutado si la condición es falsa (false)  
}
```

Anidamiento: Se pueden encadenar condiciones sucesivas mediante **else if**:

```
if (nota >= 9) {  
    std::cout << "Sobresaliente";  
} else if (nota >= 7) {  
    std::cout << "Notable";  
} else {  
    std::cout << "Aprobado/Suspenso";  
}
```

1.2 Estructura **switch**

Es una alternativa más legible a múltiples condicionales anidadas cuando evaluamos una única variable de tipo entero, carácter o enumerado frente a valores constantes (casos):

```
switch (opcion) {  
    case 1:  
        // Código para caso 1  
        break;  
    case 2:  
        // Código para caso 2  
        break;  
    default:  
        // Código si no coincide con ningún caso previo  
}
```

[!WARNING] El uso de la instrucción **break** al final de cada caso es fundamental. Si se omite, la ejecución continuará en los siguientes casos de forma secuencial (fenómeno conocido como *fallthrough*), lo cual suele generar errores de lógica indeseados.

2. Estructuras Iterativas (Bucles)

Los bucles permiten repetir la ejecución de un bloque de código mientras se cumpla una condición dada.

2.1 Bucle **while** (Bucle con Pre-condición)

Evalúa la condición **antes** de ejecutar el cuerpo del bucle. Si la condición inicial es falsa, el bucle nunca se llega a ejecutar:

```
while (condicion) {  
    // Código a repetir  
    // (Debe alterar alguna variable de la condición para evitar un bucle infinito)  
}
```

2.2 Bucle **do-while** (Bucle con Post-condición)

Ejecuta el cuerpo del bucle **al menos una vez**, y luego evalúa la condición de repetición al final:

```
do {  
    // Código ejecutado al menos una vez y repetido mientras se cumpla la condición  
} while (condicion);
```

Es ideal para validación de entradas del usuario y menús de consola.

2.3 Bucle **for** (Bucle Controlado por Contador)

Es el más estructurado y compacto para repetir un bloque de código un número conocido de veces:

```
for (inicializacion; condicion; incremento) {  
    // Código a repetir  
}
```

Equivalencia: Un bucle **for** se puede escribir equivalentemente como un bucle **while**:

```
inicializacion;  
while (condicion) {  
    // Código  
    incremento;  
}
```

3. Instrucciones de Control de Bucles: **break** y **continue**

C++ proporciona dos palabras clave para alterar el comportamiento ordinario de los bucles:

- **break**: Finaliza inmediatamente la ejecución del bucle, saltando a la primera instrucción que haya fuera de este.
- **continue**: Omite el resto de la iteración actual y salta de inmediato a la siguiente evaluación de la condición del bucle (o incremento en un bucle **for**).

4. El Toque Informático

Eficiencia, Anidamiento e Inestabilidad de Bucles

1. **Bucle Infinito**: Si la condición de un bucle nunca pasa a ser falsa (por ejemplo, `while(true)` sin un **break** interno, o decremento erróneo en una condición de parada), el programa entrará en un bucle que consumirá el 100% de uso de la CPU, congelando el proceso.
2. **Anidamiento y Complejidad**: Anidar bucles (un bucle dentro de otro) eleva drásticamente la cantidad de operaciones. Si un bucle exterior hace n repeticiones y el interior hace n repeticiones, el código del interior se ejecutará n^2 veces (complejidad cuadrática $O(n^2)$). Los programadores deben intentar limitar la anidación profunda para mantener los programas rápidos y optimizados.

A continuación, implementamos un programa interactivo en C++ que valida la entrada del usuario mediante un menú interactivo usando un bucle **do-while** y un selector **switch**.

```
#include <iostream>

int main() {
    int opcion;

    // Bucle do-while para mantener el menú activo hasta que se elija salir (opción 3)
    do {
        std::cout << "\n=== MENU INTERACTIVO ===" << std::endl;
        std::cout << "1. Mostrar saludo" << std::endl;
        std::cout << "2. Mostrar consejo de programacion" << std::endl;
        std::cout << "3. Salir del programa" << std::endl;
        std::cout << "Elige una opcion (1-3): ";
```

```
std::cin >> opcion;

// Estructura condicional múltiple switch
switch (opcion) {
    case 1:
        std::cout << "¡Hola! Bienvenido al manual de Programacion I." <<
std::endl;
        break;
    case 2:
        std::cout << "Consejo: Evita anidar demasiados bucles para mantener
tu codigo eficiente." << std::endl;
        break;
    case 3:
        std::cout << "Saliendo del programa..." << std::endl;
        break;
    default:
        std::cout << "Opcion no valida. Por favor, introduce un numero del
1 al 3." << std::endl;
}
} while (opcion != 3);

return 0;
}
```

5. Ejercicios Resueltos

Ejercicio 1

Escribir un programa en C++ que sume los números del 1 al 100 usando un bucle **for** y muestre el resultado por pantalla.

Solución:

```
#include <iostream>

int main() {
    int suma = 0; // Acumulador inicializado en cero

    // El bucle inicializa i en 1, repite mientras i <= 100 e incrementa i en 1 en
    cada paso
    for (int i = 1; i <= 100; ++i) {
        suma += i; // Equivalente a: suma = suma + i
    }

    std::cout << "La suma de los numeros del 1 al 100 es: " << suma << std::endl;
}
```

```
    return 0;  
}
```

(Nota matemática: La suma se podría calcular de forma directa y más eficiente en $O(1)$ operaciones mediante la fórmula de Gauss: $S = \frac{n(n+1)}{2} = \frac{100 \cdot 101}{2} = 5050$).

6. Ejercicios Propuestos

1. Escribir un programa en C++ que solicite una calificación numérica (0 a 10) al usuario y muestre por pantalla si está "Aprobado" (nota ≥ 5) o "Suspenso" (nota < 5) utilizando tanto un `if` como un operador ternario (`? :`).
2. Escribir un programa en C++ que imprima en la consola los primeros n números primos, donde n lo introduce el usuario.
3. ¿Qué ocurre si se omite el incremento en un bucle `for`? ¿Produce esto un error de compilación o de ejecución?

Tema 4: Desarrollo Modular

A medida que los programas crecen en complejidad, escribir todo el código dentro de la función `main` se vuelve insostenible. El **desarrollo modular** consiste en dividir un problema complejo en subproblemas más pequeños y fáciles de resolver de forma aislada (metodología "divide y vencerás" o **Diseño Descendente**). Cada módulo se implementa mediante una **función** autónoma.

1. Declaración y Definición de Funciones

En C++, una función debe ser conocida por el compilador antes de poder ser invocada. Por ello, se divide en dos fases:

1.1 Declaración (Prototipo)

Informa al compilador sobre la existencia de la función, su nombre, qué parámetros recibe y qué tipo de dato devuelve. Se sitúa habitualmente al principio del archivo (o en archivos de cabecera `.h`):

```
tipo_retorno nombre_funcion(tipo_parametro1 param1, tipo_parametro2 param2);
```

1.2 Definición

Contiene el cuerpo de la función con las instrucciones a ejecutar. Se suele implementar debajo de la función `main`:

```
tipo_retorno nombre_funcion(tipo_parametro1 param1, tipo_parametro2 param2) {  
    // Código de la función  
    return valor_retorno; // Si el tipo_retorno no es void  
}
```

Funciones sin retorno (void): Si una función realiza una tarea (como imprimir un menú o inicializar un recurso) pero no devuelve ningún valor de cálculo, su tipo de retorno se declara como `void` y la instrucción `return` es opcional.

2. Paso de Parámetros: Valor vs Referencia

Los parámetros son las variables de entrada que una función necesita para operar. C++ ofrece dos mecanismos fundamentales para pasar argumentos a una función:

2.1 Paso por Valor

El compilador realiza una **copia** del valor de la variable original en el parámetro formal de la función.

- **Comportamiento:** Cualquier modificación que se realice sobre el parámetro dentro de la función **no afecta** a la variable original del programa llamador.
- **Uso:** Es seguro y el predeterminado para tipos de datos primitivos pequeños (`int`, `float`, `char`).

2.2 Paso por Referencia

Se pasa la **dirección de memoria** de la variable original utilizando el operador `&` en la declaración del parámetro.

- **Comportamiento:** El parámetro formal actúa como un alias (referencia directa) de la variable original. Cualquier modificación que se realice dentro de la función **alterará directamente** la variable original.
 - **Uso:** Permite que una función devuelva múltiples resultados modificando las variables pasadas por referencia, y evita la copia ineficiente en memoria de estructuras u objetos pesados.
-

3. Especificación de Funciones: Precondiciones y Postcondiciones

El diseño de software robusto se basa en el **Diseño por Contrato** mediante la definición formal de precondiciones y postcondiciones:

- **Precondiciones (Pre):** Requisitos que deben cumplirse obligatoriamente antes de invocar la función (por ejemplo, "el divisor de una división no puede ser cero" o "el valor de entrada de una raíz cuadrada debe ser no negativo"). Es responsabilidad de quien invoca a la función asegurar que se cumplan.
 - **Postcondiciones (Post):** Garantías que la función asegura cumplir al finalizar su ejecución, siempre y cuando se hayan satisfecho las precondiciones (por ejemplo, "la función devuelve el valor absoluto de la entrada" o "el fichero de salida ha sido guardado en disco").
-

4. El Toque Informático

Gestión de Memoria: La Pila de Llamadas (Call Stack)

Cuando un programa invoca a una función, el sistema operativo reserva un bloque de memoria temporal en la **Pila de Llamadas** (Call Stack) conocido como **Marco de Activación** (Stack Frame). El Marco de Activación contiene:

1. Las variables locales y parámetros formales de la función.
2. La **dirección de retorno** (dónde debe continuar el programa principal cuando termine la función).

Rendimiento en memoria:

- Si pasamos una variable por **valor**, se reservan bytes adicionales en el Stack Frame para almacenar su copia. Si la estructura a copiar es grande, esto puede provocar saturación de memoria.
- Si la pasamos por **referencia**, solo se almacena un puntero (típicamente de 4 u 8 bytes) al Stack Frame de la función llamadora, optimizando el consumo de

recursos.

A continuación, implementamos el clásico algoritmo de intercambio de variables (**swap**) en C++ para ilustrar el comportamiento del paso por valor frente al paso por referencia.

```
#include <iostream>

// 1. Paso por Valor: Intenta intercambiar pero falla al operar sobre copias
void swap_por_valor(int a, int b) {
    int aux = a;
    a = b;
    b = aux;
}

// 2. Paso por Referencia: Utiliza el operador & para modificar las variables originales
void swap_por_referencia(int& a, int& b) {
    int aux = a;
    a = b;
    b = aux;
}

int main() {
    int x = 10;
    int y = 20;

    std::cout << "Valores iniciales: x = " << x << ", y = " << y << std::endl;

    // Invocación por valor
    swap_por_valor(x, y);
    std::cout << "Valores tras swap_por_valor: x = " << x << ", y = " << y << "
(Fallo)" << std::endl;

    // Invocación por referencia
    swap_por_referencia(x, y);
    std::cout << "Valores tras swap_por_referencia: x = " << x << ", y = " << y << "
(Exito)" << std::endl;

    return 0;
}
```

5. Ejercicios Resueltos

Ejercicio 1

Dada la siguiente función, documentar sus Precondiciones y Postcondiciones y explicar qué devuelve.

```
double calcular_raiz_division(double dividendo, double divisor) {  
    return sqrt(dividendo / divisor);  
}
```

Solución (Especificación de Contrato):

- **Precondiciones:**

1. El **divisor** debe ser distinto de cero ($divisor \neq 0$) para evitar una división por cero en coma flotante.
2. El cociente de la división must ser mayor o igual a cero ($\frac{dividendo}{divisor} \geq 0$) para que la raíz cuadrada real esté matemáticamente definida.

- **Postcondiciones:**

1. La función devuelve la raíz cuadrada del cociente de la división entre el dividendo y el divisor, expresada como un número real de precisión doble (**double**).
2. Las variables originales **dividendo** y **divisor** no sufren modificaciones en el programa llamador ya que se pasaron por valor.

6. Ejercicios Propuestos

1. Escribir una función en C++ llamada **calcular_potencia** que tome una base real (**double**) y un exponente entero (**int**) y devuelva el resultado de elevar la base a la potencia. (Define sus firmas/declaraciones y definición).
2. Escribir una función que reciba el radio de una esfera y devuelva tanto su área superficial como su volumen a través de paso de parámetros por referencia.
3. ¿Qué es un desbordamiento de pila (*stack overflow*) y cómo se relaciona con la pila de llamadas al invocar funciones de forma recursiva infinita?

Tema 5: Vectores y Matrices (Arrays)

Los tipos de datos primitivos individuales (como `int` o `double`) solo pueden almacenar un valor a la vez. Un **array** (o arreglo/vector) es una estructura de datos estática y homogénea que almacena una colección secuencial de elementos del mismo tipo bajo un único nombre. Su acceso directo en tiempo constante ($O(1)$) lo convierte en la estructura fundamental de almacenamiento de información lineal y matricial.

1. Definición y Uso de Arrays en C++

Los arrays reservan un bloque de memoria contiguo en el sistema.

1.1 Arrays Unidimensionales (Vectores)

Se declaran indicando el tipo, nombre del array y su tamaño (que debe ser una constante entera conocida en tiempo de compilación):

```
int temperaturas[7]; // Array de 7 enteros
```

Indexación: Los elementos se acceden mediante índices enteros que van desde `0` hasta `tamaño - 1`.

```
temperaturas[0] = 22; // Primer elemento  
temperaturas[6] = 18; // Séptimo (último) elemento
```

[!CAUTION] C++ **no comprueba los límites del array** en tiempo de ejecución. Acceder a un índice fuera del rango (por ejemplo, `temperaturas[7]`) provocará un error de segmentación (Segmentation Fault) o leerá basura de memoria, desestabilizando el programa.

1.2 Arrays Bidimensionales (Matrices)

Se declaran con dos conjuntos de corchetes representing filas y columnas:

```
int matriz[3][4]; // Matriz de 3 filas y 4 columnas
```

Se accede mediante dos índices: `matriz[fila][columna]`.

```
matriz[0][0] = 5; // Elemento en la esquina superior izquierda
```

2. Algoritmos de Búsqueda

La búsqueda consiste en encontrar la posición de un elemento (clave) dentro de un array.

2.1 Búsqueda Lineal (Secuencial)

Recorre el array desde el principio hasta el final, comparando cada elemento con la clave buscada.

- **Complejidad:** $O(n)$ en el peor de los casos (si el elemento no existe o está al final).
- **Requisito:** Funciona en cualquier array (ordenado o desordenado).

2.2 Búsqueda Binaria (Dicotómica)

Divide repetidamente a la mitad el rango de búsqueda. Compara la clave con el elemento central; si no coincide, reduce el rango al sub-array izquierdo o derecho.

- **Complejidad:** $O(\log n)$ (extremadamente rápida, análoga a la búsqueda en un diccionario).
- **Requisito:** El array debe estar ordenado previamente.

3. Algoritmos de Ordenación

La ordenación consiste en organizar los elementos del array en un orden lógico (normalmente ascendente).

1. **Ordenación por Burbuja (Bubble Sort):** Compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Repite el proceso hasta que no se requieran más intercambios.
 - *Complejidad:* $O(n^2)$
 2. **Ordenación por Selección (Selection Sort):** Busca el elemento más pequeño del array y lo intercambia con el elemento de la primera posición. Luego busca el menor del resto del array y lo intercambia con la segunda posición, y así sucesivamente.
 - *Complejidad:* $O(n^2)$
 3. **Ordenación por Inserción (Insertion Sort):** Toma cada elemento y lo inserta en su posición correcta respecto a los elementos ya ordenados a su izquierda (similar a cómo se ordenan las cartas en la mano).
 - *Complejidad:* $O(n^2)$
-

4. El Toque Informático

Búsqueda Binaria en Grandes Volúmenes de Datos

Para un array de 1 millón de elementos:

- La **Búsqueda Lineal** requiere hasta 1 millón de comparaciones.
- La **Búsqueda Binaria** requiere a lo sumo $\log_2(1000000) \approx 20$ comparaciones.

Este enorme ahorro de tiempo ilustra por qué en ingeniería informática es preferible incurrir en el coste de ordenar un array una sola vez (mediante algoritmos rápidos como QuickSort o MergeSort de $O(n \log n)$) para después realizar miles de búsquedas binarias instantáneas de $O(\log n)$.

A continuación, implementamos en C++ el algoritmo de ordenación por selección para ordenar un vector, y posteriormente realizamos una búsqueda binaria en él.

```
#include <iostream>
```

```

// Función para ordenar un array usando Selección Directa (Selection Sort)
void ordenar_seleccion(int arr[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        int idx_minimo = i;
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[idx_minimo]) {
                idx_minimo = j;
            }
        }
        // Intercambiamos el menor encontrado con el elemento en la posición i
        int aux = arr[i];
        arr[i] = arr[idx_minimo];
        arr[idx_minimo] = aux;
    }
}

// Función que realiza la Búsqueda Binaria (O(log n))
int busqueda_binaria(const int arr[], int n, int clave) {
    int izquierda = 0;
    int derecha = n - 1;

    while (izquierda <= derecha) {
        int medio = izquierda + (derecha - izquierda) / 2;

        if (arr[medio] == clave) {
            return medio; // Elemento encontrado, devolvemos su índice
        }
        if (arr[medio] < clave) {
            izquierda = medio + 1; // Descartamos la mitad izquierda
        } else {
            derecha = medio - 1; // Descartamos la mitad derecha
        }
    }
    return -1; // Elemento no encontrado
}

int main() {
    const int TAM = 8;
    int datos[TAM] = {34, 12, 5, 89, 56, 21, 8, 77};

    std::cout << "Vector original desordenado: ";
    for (int i = 0; i < TAM; ++i) std::cout << datos[i] << " ";
    std::cout << std::endl;

    // 1. Ordenamos el vector (requisito para búsqueda binaria)
    ordenar_seleccion(datos, TAM);

    std::cout << "Vector ordenado por seleccion: ";
    for (int i = 0; i < TAM; ++i) std::cout << datos[i] << " ";
    std::cout << std::endl;

    // 2. Realizamos la búsqueda
    int buscar = 21;
    int posicion = busqueda_binaria(datos, TAM, buscar);

    if (posicion != -1) {
        std::cout << "El elemento " << buscar << " se encuentra en el índice: " <<

```

```
posicion << std::endl;
    } else {
        std::cout << "El elemento " << buscar << " no esta en el vector." <<
std::endl;
    }

    return 0;
}
```

5. Ejercicios Resueltos

Ejercicio 1

Escribir una función en C++ que reciba una matriz de tamaño 3×3 de enteros y devuelva la suma de todos los elementos situados en su diagonal principal (traza de la matriz).

Solución:

```
#include <iostream>

const int FILAS = 3;
const int COLUMNAS = 3;

// La diagonal principal solo existe en matrices cuadradas (fila == columna)
int calcular_traza(const int matriz[FILAS][COLUMNAS]) {
    int suma = 0;
    for (int i = 0; i < FILAS; ++i) {
        suma += matriz[i][i]; // Acceso directo al elemento diagonal
    }
    return suma;
}

int main() {
    int M[FILAS][COLUMNAS] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    int traza = calcular_traza(M);
    std::cout << "La traza de la matriz (suma diagonal) es: " << traza <<
std::endl; // 1 + 5 + 9 = 15
    return 0;
}
```

6. Ejercicios Propuestos

1. Escribir una función que reciba un array de enteros y devuelva tanto el valor máximo como el valor mínimo presentes en el mismo.
2. Implementar la función de búsqueda lineal en C++ y realizar una traza de ejecución comparando el número de iteraciones que realiza frente a la búsqueda binaria para encontrar el número 89 en el array de ejemplo del Tema 5.
3. ¿Por qué los arrays en C++ se pasan siempre por referencia de forma implícita al invocarse en funciones? ¿Qué implicaciones tiene esto en la gestión de memoria y el rendimiento?

Tema 6: Cadenas de Caracteres

La manipulación de texto es uno de los campos de aplicación más extendidos de la informática. Una **cadena de caracteres** es una secuencia ordenada de caracteres. En C++, coexisten dos formas de representar cadenas: las tradicionales cadenas al estilo del lenguaje C (C-strings) y los objetos modernos `std::string` de la biblioteca estándar de C++.

1. C-Strings (Cadenas al estilo de C)

En el lenguaje C, una cadena de caracteres no es un tipo de dato nativo, sino un **array de tipo `char`** que finaliza obligatoriamente con el carácter nulo (`'\0'`).

```
Cadena: "H" "o" "l" "a" "\0"  
Índice: 0  1  2  3  4
```

El compilador reserva un byte adicional automáticamente para almacenar el carácter terminador `'\0'`, que indica el fin de la lectura de la cadena.

Funciones de la biblioteca estándar <cstring>

Para operar con C-strings, se deben utilizar funciones de la librería de C:

- `strlen(cadena)`: Devuelve la longitud real del texto (sin contar el `'\0'`).
- `strcpy(destino, origen)`: Copia el contenido de una cadena en otra.
- `strcat(destino, origen)`: Concatena (une) la cadena de origen al final del destino.
- `strcmp(cad1, cad2)`: Compara alfabéticamente dos cadenas. Devuelve `0` si son idénticas, un valor negativo si `cad1 < cad2` y positivo en caso contrario.

[!CAUTION] **Vulnerabilidad de Buffer Overflow (Desbordamiento de Búfer):**

Funciones como `strcpy` o `strcat` no comprueban si el array de destino tiene espacio suficiente para albergar la nueva cadena. Si es más grande, se

sobrescribirán posiciones de memoria contiguas a la variable. Esto constituye una de las mayores brechas de seguridad informática históricas, explotada por malware para inyectar código malicioso en el sistema.

2. La clase `std::string` de C++ Moderno

Para evitar los riesgos y la rigidez de las C-strings, C++ proporciona la clase `std::string` dentro de la biblioteca estándar `<string>`. Este tipo de datos gestiona de forma dinámica y automática la memoria necesaria.

2.1 Declaración y Operaciones Básicas

- **Asignación directa:**

```
std::string mensaje = "Hola Mundo";
```

- **Concatenación mediante operador `+`:**

```
std::string saludo = "Hola " + nombre;
```

- **Comparación directa:** Se usan los operadores relacionales estándar (`==`, `!=`, `<`, `>`).

```
if (cad1 == cad2) { /* ... */ }
```

2.2 Métodos Útiles de `std::string`

- `size()` o `length()`: Devuelven el número de caracteres de la cadena.
- `empty()`: Devuelve `true` si la cadena está vacía.
- `clear()`: Vacía la cadena.

- `substr(posicion, longitud)`: Devuelve una subcadena a partir de la posición indicada y con el tamaño solicitado.
 - `find(subcadena)`: Busca la primera aparición de una subcadena y devuelve su índice; si no la encuentra, devuelve la constante `std::string::npos`.
-

3. El Toque Informático

Lectura Robusta de Cadenas con Espacios

El operador de extracción estándar `cin >> cadena` lee texto de la consola, pero **se detiene al encontrar el primer espacio en blanco, tabulador o salto de línea**. Si queremos leer una frase completa con espacios (por ejemplo, un nombre completo o dirección):

- Para `std::string`, usamos la función `getline(cin, cadena)`.
- Para C-strings, usamos `cin.getline(array, tamaño)`.

A continuación, implementamos en C++ un programa que procesa una cadena de texto (calculando estadísticas e identificando palabras) empleando las facilidades del tipo `std::string`.

```
#include <iostream>
#include <string>

int main() {
    std::string nombre_completo;

    std::cout << "Introduce tu nombre completo: ";
    // Leemos la linea entera incluyendo los espacios
    std::getline(std::cin, nombre_completo);

    std::cout << "\nEstadísticas del nombre:" << std::endl;
    std::cout << "Texto completo: " << nombre_completo << std::endl;
    std::cout << "Numero de caracteres (con espacios): " <<
nombre_completo.length() << std::endl;
    std::cout << "¿Esta vacio?: " << (nombre_completo.empty() ? "Si" : "No") <<
std::endl;

    // Buscar la presencia de un espacio en blanco para separar el primer nombre
    size_t indice_espacio = nombre_completo.find(' ');

    if (indice_espacio != std::string::npos) {
        // Extraemos la subcadena que va desde la posición 0 hasta el espacio
```

```
        std::string primer_nombre = nombre_completo.substr(0, indice_espacio);
        std::cout << "Primer nombre extraído: " << primer_nombre << std::endl;
    } else {
        std::cout << "No se han detectado espacios. Nombre de una sola palabra." <<
std::endl;
    }

    return 0;
}
```

4. Ejercicios Resueltos

Ejercicio 1

Escribir una función en C++ llamada `es_palindromo` que tome un `std::string` y devuelva un valor booleano indicando si la palabra se lee igual de izquierda a derecha que de derecha a izquierda (ignorando mayúsculas/minúsculas).

Solución:

```
#include <iostream>
#include <string>
#include <cctype> // Contiene la función tolower

bool es_palindromo(const std::string& cad) {
    int izq = 0;
    int der = cad.length() - 1;

    while (izq < der) {
        // Convertimos a minúsculas antes de comparar para ser tolerantes al caso
        if (std::tolower(cad[izq]) != std::tolower(cad[der])) {
            return false; // No coincide, no es palíndromo
        }
        izq++;
        der--;
    }
    return true; // Ha coincidido toda la palabra
}

int main() {
    std::string palabra = "Radar"; // 'R' y 'r' coincidirán gracias a tolower

    if (es_palindromo(palabra)) {
        std::cout << palabra << " es un palindromo." << std::endl;
    } else {
        std::cout << palabra << " no es un palindromo." << std::endl;
    }
}
```

```
    return 0;  
}
```

5. Ejercicios Propuestos

1. Escribir un programa en C++ que lea una frase del usuario y cuente el número de vocales (a, e, i, o, u) presentes en ella.
2. ¿Qué diferencia hay en el comportamiento en memoria del paso de un `std::string` por valor frente a pasarlo por referencia constante (`const std::string&`) en las funciones?
3. Describir qué es el carácter de terminación nula `'\0'`, y explicar qué ocurriría si un array de caracteres que representa a "Hola" careciera de este terminador al imprimirse con `std::cout`.

Tema 7: Registros (Structs)

Las estructuras de datos homogéneas (como vectores y matrices) solo pueden almacenar elementos del mismo tipo. Para modelar entidades del mundo real que constan de múltiples atributos de diferente naturaleza (por ejemplo, los datos de un cliente: nombre, edad, saldo, si está activo), se recurre a las estructuras heterogéneas conocidas en programación estructurada como **Registros** o **Estructuras** (**struct** en C++).

1. Definición y Acceso a Miembros

Un **registro** es un tipo de dato personalizado que agrupa bajo un mismo nombre una colección de variables (denominadas campos o miembros) que pueden tener tipos distintos.

Definición de un **struct**

Se define fuera de las funciones utilizando la palabra clave **struct** seguida del nombre de la estructura y de sus campos encerrados entre llaves (finalizando obligatoriamente con un punto y coma **;**):

```
struct Cliente {  
    std::string nombre;  
    int edad;  
    double saldo;  
    bool activo;  
};
```

Declaración e Inicialización de Variables

Una vez definida la estructura, se puede utilizar como cualquier tipo de dato primitivo para declarar variables:

```
Cliente c1;  
c1.nombre = "Luis Felipe";  
c1.edad = 25;  
c1.saldo = 1250.50;  
c1.activo = true;
```

El operador punto (.): Se utiliza para acceder individualmente a cada uno de los miembros o campos de la estructura.

2. Anidamiento de Estructuras

Un miembro de un `struct` puede ser a su vez otra estructura previamente definida:

```
struct Fecha {  
    int dia;  
    int mes;  
    int anio;  
};  
  
struct Empleado {  
    std::string nombre;  
    double salario;  
    Fecha fecha_contratacion; // Estructura anidada  
};
```

Para acceder a los campos anidados, se encadenan los operadores punto:

```
Empleado emp;  
emp.nombre = "Ana";  
emp.fecha_contratacion.dia = 15;  
emp.fecha_contratacion.mes = 6;  
emp.fecha_contratacion.anio = 2026;
```

3. Combinación de Arrays y Estructuras

La potencia de los registros se multiplica al combinarlos con arrays.

3.1 Estructuras con Arrays Internos

Un registro puede contener arrays como miembros (por ejemplo, un estudiante con sus calificaciones):

```
struct Estudiante {  
    std::string nombre;  
    double notas[5]; // Array interno de notas  
};
```

3.2 Arrays de Estructuras (Bases de Datos en Memoria)

Podemos declarar vectores cuyos elementos sean de tipo `struct` para almacenar bases de datos tabulares completas en la memoria RAM:

```
const int MAX_ALUMNOS = 100;  
Estudiante clase[MAX_ALUMNOS]; // Array de 100 estructuras Estudiante
```

4. El Toque Informático

Alineación de Memoria y Padding de Estructuras

El tamaño en memoria de un `struct` no es necesariamente la suma exacta de los tamaños de sus miembros. Las CPUs modernas leen datos de la memoria RAM en bloques alineados a palabras (normalmente de 4 u 8 bytes). Para acelerar la velocidad de acceso, el compilador introduce bytes de relleno invisibles (**padding**) entre los campos de forma que cada variable comience en una dirección de memoria múltiplo de su tamaño. Por ejemplo:

```
struct Ineficiente {  
    char a; // 1 byte
```

```
// 3 bytes de padding para alinear el int a una dirección múltiplo de 4
int b;    // 4 bytes
char c;   // 1 byte
// 3 bytes de padding al final
}; // sizeof es 12 bytes, en lugar de 6 bytes.
```

Optimización: Organizar los campos de mayor a menor tamaño (colocando `int` antes de `char`) minimiza el padding introducido por el compilador, optimizando el uso de la memoria RAM.

A continuación, implementamos en C++ un sistema de gestión escolar básico (base de datos en memoria) utilizando arrays de estructuras.

```
#include <iostream>
#include <string>

// Definimos la estructura del Alumno
struct Alumno {
    std::string nombre;
    int matricula;
    double nota_final;
};

// Función para imprimir los datos de un alumno (paso por referencia constante para eficiencia)
void mostrar_alumno(const Alumno& al) {
    std::cout << "Nombre: " << al.nombre
               << " | Mat: " << al.matricula
               << " | Nota: " << al.nota_final << std::endl;
}

// Función para buscar a un alumno por su número de matrícula (búsqueda lineal)
int buscar_por_matricula(const Alumno grupo[], int size, int mat_buscada) {
    for (int i = 0; i < size; ++i) {
        if (grupo[i].matricula == mat_buscada) {
            return i; // Devolvemos la posición en el array
        }
    }
    return -1; // No encontrado
}

int main() {
    const int N = 3;
    // Declaración e inicialización directa de un array de estructuras Alumno
    Alumno clase[N] = {
        {"Juan Perez", 1001, 8.5},
        {"Maria Gomez", 1002, 9.2},
        {"Carlos Ruiz", 1003, 4.7}
    };

    std::cout << "--- Listado de Alumnos ---" << std::endl;
    for (int i = 0; i < N; ++i) {
```

```

        mostrar_alumno(clase[i]);
    }

    // Búsqueda
    int buscar_mat = 1002;
    std::cout << "\nBuscando matricula " << buscar_mat << "..." << std::endl;
    int pos = buscar_por_matricula(clase, N, buscar_mat);

    if (pos != -1) {
        std::cout << "¡Alumno encontrado!" << std::endl;
        mostrar_alumno(clase[pos]);
    } else {
        std::cout << "No existe ningun alumno con esa matricula." << std::endl;
    }

    return 0;
}

```

5. Ejercicios Resueltos

Ejercicio 1

Definir una estructura en C++ llamada **Fraccion** que represente un número fraccionario (con numerador y denominador enteros). Escribir una función que tome dos estructuras **Fraccion** y devuelva una nueva estructura **Fraccion** que corresponda a la suma de ambas.

Solución:

```

#include <iostream>

struct Fraccion {
    int numerador;
    int denominador;
};

// Función de suma de fracciones: (a/b) + (c/d) = (ad + bc) / bd
Fraccion sumar_fracciones(const Fraccion& f1, const Fraccion& f2) {
    Fraccion resultado;
    resultado.numerador = f1.numerador * f2.denominador + f2.numerador *
f1.denominador;
    resultado.denominador = f1.denominador * f2.denominador;
    return resultado;
}

int main() {

```

```
Fraccion fr1 = {1, 2}; // 1/2
Fraccion fr2 = {1, 3}; // 1/3

Fraccion suma = sumar_fracciones(fr1, fr2);

std::cout << "Suma: " << suma.numerador << "/" << suma.denominador <<
std::endl; // 5/6
return 0;
}
```

6. Ejercicios Propuestos

1. Escribir una estructura llamada **Punto2D** con coordenadas **x** e **y** (**double**), y desarrollar una función que calcule y devuelva la distancia euclidiana entre dos puntos.
2. ¿Qué diferencia hay en el paso de un **struct** a una función por valor frente a pasarlo por referencia constante? Explica el impacto en la memoria Stack.
3. Utilizando el operador **sizeof**, escribe un programa en C++ que compare el tamaño de una estructura vacía frente a una con campos para ver cuántos bytes le asigna el compilador.

Tema 8: Flujos y Ficheros

La memoria principal (RAM) del ordenador es volátil y tiene una capacidad limitada. Al cerrar el programa o apagar el equipo, todas las variables, arrays y estructuras se pierden. Para lograr que la información se conserve a largo plazo se requiere de la **persistencia de datos** en soportes de almacenamiento secundario (discos duros y SSDs) utilizando **ficheros**. En C++, esto se modela a través del concepto de **flujo** (stream).

1. El Concepto de Flujo (Stream) en C++

Un **flujo** es una abstracción lógica que representa un canal de comunicación unidireccional para transferir datos de forma secuencial desde una fuente (productor) hasta un sumidero (consumidor).

C++ utiliza la biblioteca estándar `<iostream>` para gestionar dos flujos básicos:

- `std::cin`: Flujo de entrada estándar (normalmente asociado al teclado).
- `std::cout`: Flujo de salida estándar (asociado a la pantalla de la consola).

Para interactuar con ficheros físicos en disco, se utiliza la biblioteca estándar `<fstream>` (File Stream), que proporciona tres tipos de flujos especializados:

- `std::ifstream`: Flujo para operaciones de **lectura** de ficheros (Input File Stream).
 - `std::ofstream`: Flujo para operaciones de **escritura** de ficheros (Output File Stream).
 - `std::fstream`: Flujo bidireccional para **lectura y escritura**.
-

2. Escritura de Ficheros de Texto (`ofstream`)

Para escribir datos en un fichero de texto, seguimos el siguiente protocolo secuencial:

1. **Declarar el flujo:** Crear un objeto del tipo `std::ofstream`.
2. **Abrir el fichero:** Vincular el flujo con un archivo en disco mediante el método `.open()`.
3. **Verificar la apertura:** Comprobar si el archivo se abrió correctamente (por ejemplo, si tenemos permisos de escritura).
4. **Escribir los datos:** Enviar información al flujo utilizando el operador de inserción `<<` (de forma idéntica a `cout`).
5. **Cerrar el flujo:** Invocar al método `.close()` para liberar el descriptor del archivo y asegurar el guardado de datos.

```
#include <fstream>

std::ofstream archivo;
archivo.open("datos.txt");
if (archivo.is_open()) {
    archivo << "Línea de texto a guardar" << std::endl;
    archivo.close();
}
```

3. Lectura de Ficheros de Texto (`ifstream`)

Para leer la información guardada en un fichero, el protocolo es análogo:

1. **Declarar el flujo:** Crear un objeto del tipo `std::ifstream`.
2. **Abrir el fichero:** Invocar `.open()`.
3. **Verificar la apertura:** Asegurar que el archivo existe en el disco duro.
4. **Leer los datos:** Extraer información del flujo utilizando el operador `>>` o `getline()`.
5. **Cerrar el flujo:** Invocar `.close()`.

Detección de Fin de Fichero (EOF)

Para leer un archivo entero de tamaño desconocido, se utiliza un bucle que comprueba si la extracción fue exitosa o si se alcanzó el fin del fichero (`eof`). La forma más robusta es usar el propio flujo como condición del bucle:

```
std::string linea;
while (std::getline(archivo, linea)) {
    // Procesar la linea leida
}
```

4. El Toque Informático

Almacenamiento en Búfer (Buffer Flushing)

Escribir datos directamente en un disco duro o SSD físico es una operación extremadamente lenta a nivel de hardware. Para solucionarlo, el sistema operativo acumula los datos de escritura en un área de memoria RAM intermedia ultrarrápida conocida como **Búfer (Buffer)**. Cuando el búfer se llena o el programa cierra el archivo, se vuelcan todos los datos al disco de golpe.

- *Vaciado Manual*: Podemos forzar la transferencia física inmediata de los datos escribiendo `std::flush` o utilizando `std::endl` (que escribe un salto de línea `\n` y realiza un *flush* automático del búfer).

A continuación, implementamos un programa en C++ completo que escribe un listado de estudiantes (registros `struct`) en un archivo de texto, y posteriormente lo lee de nuevo en un array en memoria.

```
#include <iostream>
#include <fstream>
#include <string>

struct Estudiante {
    std::string nombre;
    int edad;
    double nota;
};

int main() {
    const std::string nombre_fichero = "alumnos.txt";

    // 1. Escritura de registros en el fichero
    std::ofstream archivo_salida;
    archivo_salida.open(nombre_fichero);

    if (!archivo_salida.is_open()) {
        std::cerr << "Error al abrir el archivo para escritura." << std::endl;
    }
}
```

```

        return 1;
    }

    // Escribimos dos estudiantes, uno por línea
    archivo_salida << "Juan_Perez 20 8.5" << std::endl;
    archivo_salida << "Maria_Gomez 22 9.2" << std::endl;

    archivo_salida.close();
    std::cout << "Datos guardados en " << nombre_fichero << " con éxito." <<
std::endl;

    // 2. Lectura y reconstrucción en memoria
    std::ifstream archivo_entrada;
    archivo_entrada.open(nombre_fichero);

    if (!archivo_entrada.is_open()) {
        std::cerr << "Error al abrir el archivo para lectura." << std::endl;
        return 1;
    }

    const int MAX_ESTUDIANTES = 5;
    Estudiante grupo[MAX_ESTUDIANTES];
    int contador = 0;

    // Leemos formateado directamente: nombre, edad y nota mientras no lleguemos al
final
    while (archivo_entrada >> grupo[contador].nombre >> grupo[contador].edad >>
grupo[contador].nota) {
        contador++;
        if (contador >= MAX_ESTUDIANTES) break; // Evitamos desbordar el array
    }

    archivo_entrada.close();

    // Imprimimos los resultados cargados desde el archivo
    std::cout << "\n--- Alumnos cargados desde fichero ---" << std::endl;
    for (int i = 0; i < contador; ++i) {
        std::cout << "Nombre: " << grupo[i].nombre
            << " | Edad: " << grupo[i].edad
            << " | Nota: " << grupo[i].nota << std::endl;
    }

    return 0;
}

```

5. Ejercicios Resueltos

Ejercicio 1

Escribir un programa en C++ que lea un archivo de texto llamado **numeros.txt** que contiene un número real por línea, calcule la suma y el promedio de todos ellos y muestre los resultados en pantalla.

Solución:

```
#include <iostream>
#include <fstream>

int main() {
    std::ifstream entrada;
    entrada.open("numeros.txt");

    if (!entrada.is_open()) {
        std::cout << "No se pudo abrir el archivo numeros.txt. Asegurate de que existe." << std::endl;
        return 1;
    }

    double numero;
    double suma = 0.0;
    int cantidad = 0;

    // Bucle de lectura hasta el fin del fichero
    while (entrada >> numero) {
        suma += numero;
        cantidad++;
    }

    entrada.close();

    if (cantidad > 0) {
        std::cout << "Cantidad de numeros leidos: " << cantidad << std::endl;
        std::cout << "Suma total: " << suma << std::endl;
        std::cout << "Promedio: " << (suma / cantidad) << std::endl;
    } else {
        std::cout << "El archivo de números estaba vacío." << std::endl;
    }

    return 0;
}
```

6. Ejercicios Propuestos

1. Escribir un programa en C++ que copie el contenido de un archivo de texto llamado **origen.txt** a otro llamado **destino.txt** carácter por carácter.

2. ¿Qué diferencia hay entre abrir un archivo en modo texto clásico frente a abrirlo en modo de adición (`std::ios::app`) al instanciar un objeto `ofstream`?
3. Explicar por qué es indispensable invocar el método `.close()` al finalizar el trabajo con un fichero en C++. ¿Qué ocurre con la memoria intermedia del búfer si el programa colapsa antes de cerrarlo?

Glosario de Términos

- **Algoritmo:** Secuencia finita, ordenada e inequívoca de pasos lógicos diseñados para resolver un problema específico.
- **Ámbito (Scope):** Región del código fuente de un programa donde una variable declarada es visible y puede ser accedida por el procesador.
- **Array (Arreglo):** Estructura de datos homogénea, contigua y estática en memoria que contiene múltiples elementos del mismo tipo referenciados bajo un nombre único e índices numéricos.
- **Bucle Infinito:** Estructura de control iterativa que se repite de forma indefinida debido a que su condición de salida nunca evalúa a falsa.
- **Búfer (Buffer):** Segmento de memoria RAM utilizado para amortiguar y acelerar transferencias físicas de datos acumulando operaciones antes de enviarlas al disco duro.
- **C-String:** Cadena de caracteres clásica representada como un array de caracteres (`char[]`) delimitado por el carácter especial de fin de cadena `'\0'`.
- **Casting (Conversión):** Proceso mediante el cual el compilador o el programador fuerza la conversión de una variable de un tipo a otro (p. ej., `static_cast`).
- **Ciclo de Vida:** Conjunto de fases secuenciales (Análisis, Diseño, Programación, Pruebas y Mantenimiento) por las que pasa una aplicación de software.
- **Compilador:** Programa traductor que convierte todo el código fuente de alto nivel en un binario ejecutable en lenguaje máquina en un único proceso.
- **Desbordamiento de Búfer (Buffer Overflow):** Fallo de lógica o vulnerabilidad en la que un programa intenta escribir más datos en un array de los que este puede albergar, sobrescribiendo zonas de memoria contiguas.
- **Desbordamiento de Entero (Overflow):** Fenómeno que ocurre cuando una operación matemática excede el rango máximo de representación del tipo de dato, reiniciando su valor en el extremo opuesto del rango.
- **Diseño Descendente (Top-Down):** Metodología de modularización estructurada que consiste en simplificar un problema grande descomponiéndolo recursivamente en funciones o subrutinas independientes.
- **Flujo (Stream):** Abstracción de un canal de datos que transporta secuencialmente bytes de información desde una fuente de entrada (`std::cin`) hasta un sumidero de salida (`std::cout`).
- **Paso por Referencia:** Método de paso de parámetros a funciones que transfiere la dirección de memoria original de la variable usando `&`, permitiendo su

modificación dentro de la función y evitando la duplicación de memoria.

- **Paso por Valor:** Método que transfiere una copia del valor de la variable a la función, aislando el valor original frente a alteraciones de la subrutina.
- **Pila de Llamadas (Call Stack):** Espacio de memoria RAM que almacena la traza de ejecución de funciones, sus variables locales (marcos de activación) y direcciones de retorno.
- **Precondición:** Requisito o restricción que debe ser estrictamente verdadera antes de invocar a una función o algoritmo.
- **Postcondición:** Garantía o resultado que la función asegura entregar al programa llamador al finalizar su tarea, asumiendo que se cumplió la precondición.
- **Registro (Struct):** Tipo de dato compuesto y heterogéneo que agrupa múltiples variables de diferente naturaleza (miembros) bajo una denominación unificada.

Bibliografía Recomendada

1. **Stroustrup, B. (2013).** *The C++ Programming Language* (4th ed.). Addison-Wesley.
 - *Nota:* Escrito por el propio creador del lenguaje C++, proporciona una aproximación exhaustiva al estándar moderno y la gestión de flujos de E/S.
2. **Deitel, P., & Deitel, H. (2016).** *C++ How to Program* (10th ed.). Pearson.
 - *Nota:* Un referente académico mundial sumamente didáctico para el estudio de arrays, funciones, estructuras `struct` y algoritmos clásicos de ordenación.
3. **Kernighan, B. W., & Ritchie, D. M. (1988).** *The C Programming Language* (2nd ed.). Prentice Hall.
 - *Nota:* El texto clásico definitivo para comprender el origen de las cadenas C-strings, el formateo básico de flujos de texto y la lógica de programación estructurada.
4. **Savitch, W. (2016).** *Absolute C++* (6th ed.). Pearson.
 - *Nota:* Muy adaptado a los cursos de ingeniería informática iniciales, con excelentes explicaciones del paso de parámetros por referencia y alineación de estructuras.
5. **Joyanes Aguilar, L. (2008).** *Fundamentos de Programación: Algoritmos, Estructuras de Datos y Objetos* (4ª ed.). McGraw-Hill.
 - *Nota:* Texto imprescindible en español para dominar el pseudocódigo, el diseño Top-Down y la formulación lógica de bucles y condicionales.